

FastAPI로 만드는 생성형 AI API 문서 자동화

Groq이란?

- Groq는 생성형 AI의 추론(Inference)에 특화된 AI 반도체와 클라우드 서비스를 개발하는 미국 기업입니다.
- 기존 GPU를 사용하는 방식과 달리, 자체 개발한 LPU(Language Processing Unit)라는 AI 전용 칩을 사용하여 매우 빠른 응답 속도와 낮은 비용을 제공하는 것이 가장 큰 특징입니다.

쉽게 말하면,

OpenAI, Google, Meta는 AI 모델을 만드는 회사라면, Groq는 그 AI 모델을 빠르고 저렴하게 실행해주는 회사입니다.

Groq vs Ollama

- Groq : 클라우드에서 매우 빠르게 LLM을 실행하는 서비스
- Ollama : 내 PC에서 로컬 LLM을 쉽게 실행하는 프로그램

항목	Groq	Ollama
인터넷	필요	불필요(모델 다운로드 후)
속도	매우 빠름	PC 성능에 따라 다름
비용	무료 사용량 + 유료	무료(Open Source)
개인정보	서버로 전송	PC 내부에서만 처리
GPU 필요	✗	권장(없어도 CPU 가능)
설치	불필요(API 사용)	설치 필요
모델 다운로드	✗	✓

파일 구조

파일	역할
main.py	FastAPI 앱 생성 및 라우터 등록
settings.py	모델, 출력 폴더, 기본 OpenAPI URL 설정
core/metadata.py	Swagger 제목, 설명, 태그 메타데이터
core/security.py	Swagger Authorize 버튼과 실습용 Bearer 토큰
core/openapi.py	OpenAPI 보안 스키마 커스터마이징

- schemas: 요청/응답 데이터의 모양을 정합니다.

파일	역할
schemas/documents.py	AI 문서 생성 요청/응답 스키마
schemas/products.py	상품 등록/수정/응답 스키마
schemas/orders.py	주문 생성/응답 스키마
schemas/users.py	회원 가입/응답 스키마
schemas/common.py	공통 오류 응답 스키마

- services: 실제 일을 처리하는 로직을 담습니다.

파일	역할
services/document_workflow.py	문서 생성 흐름 조합
services/document_generator.py	Groq에 전달할 프롬프트 작성
services/groq_client.py	Groq 스트리밍 호출
services/openapi_spec.py	OpenAPI 스펙 수집 및 compact 처리
services/storage.py	생성 결과와 스냅샷 파일 저장
services/shop_store.py	예제 쇼핑몰 API의 임시 메모리 저장소

- routers: 실제 API 주소를 정의합니다.

파일	역할
<code>routers/documents.py</code>	생성형 AI 문서 자동화 API
<code>routers/health.py</code>	서버 상태 확인 API
<code>routers/products.py</code>	문서화 대상 예제 API
<code>routers/orders.py</code>	문서화 대상 예제 API
<code>routers/users.py</code>	문서화 대상 예제 API

흐름으로 보면 이렇게 이해하면 됩니다.

사용자 요청
↓
routers: 어떤 API 주소로 들어왔는지 처리
↓
schemas: 요청 데이터가 올바른지 검사
↓
services: 실제 기능 수행
↓
schemas: 응답 모양 정리
↓
사용자에게 응답

FastAPI 앱 진입점

`main.py` 가 강의의 기본 실행 파일입니다.

```
def create_app() -> FastAPI:
    docs_enabled = not is_production()

    app = FastAPI(
        title="생성형 AI API 문서 자동화",
        ...
    )

    app.include_router(health_router)
    app.include_router(products_router)
    app.include_router(orders_router)
    app.include_router(users_router)
    app.include_router(documents_router)
    app.openapi = lambda: custom_openapi(app)
    return app
```

예제 API와 AI 문서화 API

하나의 FastAPI 서버 안에 두 종류의 엔드포인트가 있습니다.

구분	경로	역할
예제 API	<code>/products</code> , <code>/orders</code> , <code>/users</code>	문서화 대상이 되는 쇼핑몰 API
AI 문서화 API	<code>/ai-docs/*</code>	OpenAPI 스펙을 Groq로 보내 문서 생성
상태 확인	<code>/health</code>	서버 상태 확인
자동 문서	<code>/docs</code>	Swagger UI
스펙 원본	<code>/openapi.json</code>	OpenAPI JSON

문서화 대상 엔드포인트

메서드	경로	기능
POST	/users	회원 가입
POST	/orders	주문 생성, 재고 차감
PATCH	/products/{product_id}	상품 부분 수정, 관리자 토큰 필요
DELETE	/products/{product_id}	상품 삭제, 관리자 토큰 필요

메서드	경로	기능
GET	/health	서버 상태 확인
GET	/products	상품 목록 조회
GET	/products/{product_id}	상품 단건 조회
GET	/orders/{order_id}	주문 조회
GET	/users/me	내 정보 조회, 관리자 토큰 필요
POST	/products	상품 등록, 관리자 토큰 필요

생성형 AI 문서화 엔드포인트

메서드	경로	생성 결과
POST	/ai-docs/api-docs	개발자용 마크다운 API 문서
POST	/ai-docs/postman-collection	Postman Collection JSON
POST	/ai-docs/readme	GitHub README 초안
POST	/ai-docs/generate-all	위 3개 문서 한 번에 생성
POST	/ai-docs/changes	변경된 엔드포인트만 문서화

Swagger 문서에 들어가는 설명

FastAPI 라우터의 `summary`, `description`, `response_model` 은 OpenAPI 스펙에 들어갑니다.

```
@router.get(
    "",
    response_model=List[ProductResponse],
    summary="상품 목록 조회",
    description="등록된 상품을 카테고리 필터와 페이지네이션으로 조회합니다.",
    response_description="상품 목록",
)
def list_products(
    category: Optional[str] = Query(default=None, description="카테고리 필터"),
    page: int = Query(default=1, ge=1, description="페이지 번호"),
    size: int = Query(default=10, ge=1, le=100, description="페이지당 항목 수"),
) -> list[dict]:
```

요청 예시는 Pydantic 모델에 작성

`json_schema_extra`에 작성한 예시는 Swagger UI와 OpenAPI 스펙에 반영됩니다.

```
class ProductCreate(BaseModel):
    model_config = ConfigDict(
        json_schema_extra={
            "examples": [
                {"name": "무선 마우스", "price": 35000, "stock": 50, "category": "전자제품"},
                {"name": "면 티셔츠", "price": 19900, "stock": 200, "category": "의류"},
            ]
        }
    )

    name: str = Field(min_length=2, max_length=100, description="상품명 (2~100자)")
    price: int = Field(gt=0, description="판매가 (원, 0 초과)")
    stock: int = Field(ge=0, default=0, description="재고 수량")
    category: str = Field(description="카테고리명")
```

인증 정보도 OpenAPI에 포함

보호된 엔드포인트는 `Authorization: Bearer demo-admin-token` 을 사용합니다.

```
bearer_scheme = HTTPBearer(auto_error=False, scheme_name="BearerAuth")

def require_admin_token(
    credentials: HTTPAuthorizationCredentials = Security(bearer_scheme),
) -> str:
    if not credentials:
        raise HTTPException(status_code=401, detail="인증이 필요합니다.")
    if credentials.credentials != DEMO_ADMIN_TOKEN:
        raise HTTPException(status_code=403, detail="관리자 권한이 필요합니다.")
    return credentials.credentials
```

OpenAPI 보안 스키마 커스터마이징

`core/openapi.py` 는 Swagger UI의 Authorize 버튼에 사용할 보안 스키마를 추가합니다.

```
components = schema.setdefault("components", {})  
security_schemes = components.setdefault("securitySchemes", {})  
security_schemes["BearerAuth"] = {  
    "type": "http",  
    "scheme": "bearer",  
    "bearerFormat": "JWT",  
}
```

전체 흐름

```
FastAPI 요청
  POST /ai-docs/api-docs
    |
    v
  GET /openapi.json
    |
    v
compact OpenAPI JSON
    |
    v
Groq API
    |
    v
output/API_DOCS.md 저장 + HTTP 응답 반환
```

OpenAPI 스펙 수집

`server_url` 에는 서버 기본 URL이나 `openapi.json` 전체 URL을 넣을 수 있습니다.

```
def build_openapi_url(base_or_spec_url: str) -> str:
    url = base_or_spec_url.rstrip("/")
    if url.endswith("/openapi.json"):
        return url
    return f"{url}/openapi.json"

def fetch_openapi_spec(url: str = DEFAULT_OPENAPI_URL) -> dict:
    spec_url = build_openapi_url(url)
    try:
        response = httpx.get(spec_url, timeout=5)
        response.raise_for_status()
        return response.json()
    except httpx.ConnectError as error:
        raise OpenAPISpecError(f"서버에 연결할 수 없습니다: {spec_url}") from error
    ...
```

프롬프트용 스펙 줄이기

OpenAPI JSON 전체를 그대로 보내면 토큰 한도를 넘기 쉽습니다.

```
def format_spec_for_prompt(spec: dict) -> str:
    compact = compact_openapi_spec(spec)
    return json.dumps(compact, ensure_ascii=False, separators=(",", ":"))
```

compact 단계에서는 다음 정보를 중심으로 남깁니다.

- API 제목, 설명, 버전
- 태그, 요약, 설명
- 경로와 실제 HTTP 메서드
- 파라미터, 요청 본문, JSON 예시
- 응답 코드와 응답 스키마 참조
- Pydantic 스키마 정의는 중복되지 않도록 한 번만 모음

Groq 호출

`services/groq_client.py` 는 `.env` 의 `GROQ_API_KEY` 를 사용합니다.

```
client = Groq()
stream = client.chat.completions.create(
    model=MODEL,
    messages=[{"role": "user", "content": prompt}],
    max_completion_tokens=max_tokens,
    reasoning_effort=reasoning_effort,
    reasoning_format="hidden",
    temperature=0.2,
    stream=True,
)
```

문서 생성 프롬프트

`services/document_generator.py` 는 생성 목적별로 다른 프롬프트를 만듭니다.

함수	생성 결과
<code>generate_api_docs()</code>	개발자용 API 문서
<code>generate_postman_collection()</code>	Postman Collection
<code>generate_readme()</code>	GitHub README
<code>generate_diff_docs()</code>	변경 엔드포인트 문서

FastAPI 라우터에서 호출

`routers/documents.py` 는 HTTP 요청을 받아 서비스 함수를 호출합니다.

```
@router.post(
    "/api-docs",
    response_model=GeneratedDocumentResponse,
    summary="API 문서 생성",
)
def generate_api_document(request: DocumentGenerationRequest) -> dict:
    try:
        return create_api_docs(request.server_url)
    except RuntimeError as error:
        raise_generation_error(error)
```

요청 본문

기본값은 현재 FastAPI 서버입니다.

```
{  
  "server_url": "http://localhost:8001"  
}
```

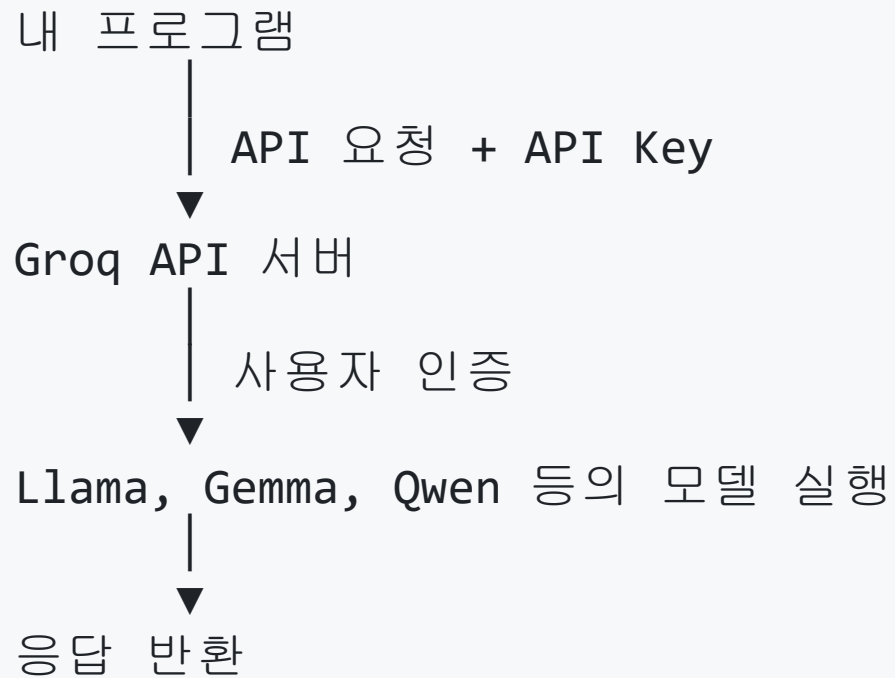
다른 FastAPI 서버를 문서화할 수도 있습니다.

```
{  
  "server_url": "http://localhost:9000/openapi.json"  
}
```

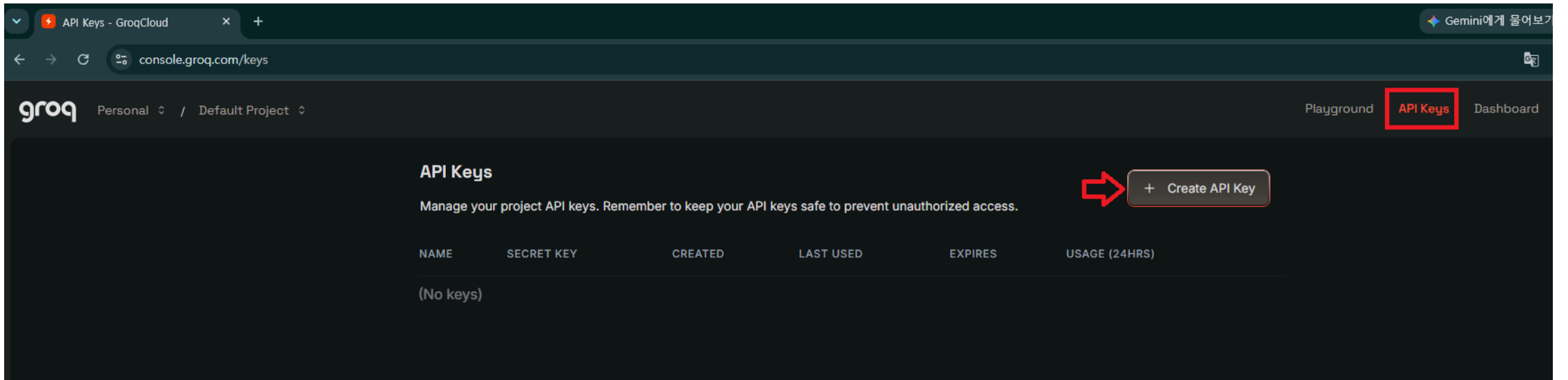
서버 실행 및 Swagger 실습

Groq API Key

- Groq API Key를 생성하는 이유는 내 프로그램이 Groq의 AI 모델을 안전하게 사용할 수 있도록 인증하기 위해서입니다.



Groq API Key 생성



Create API Key

Display Name

tutorial

A display name for the key. Maximum 50 characters.

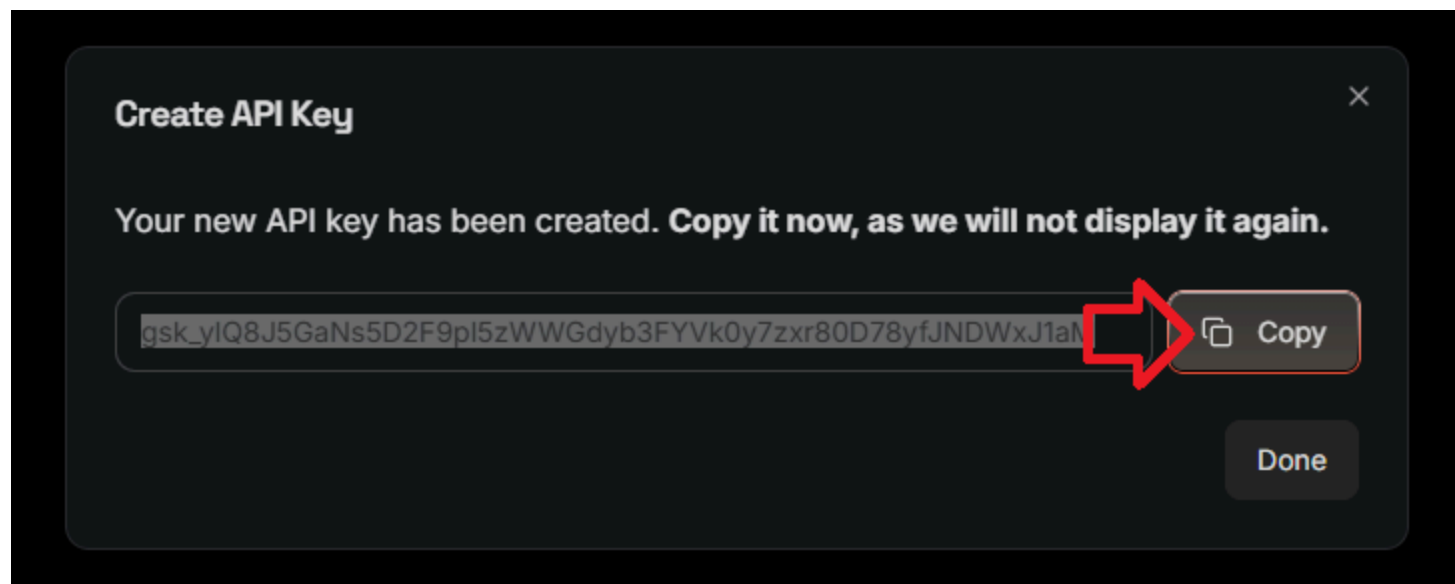
Expiration

No expiration

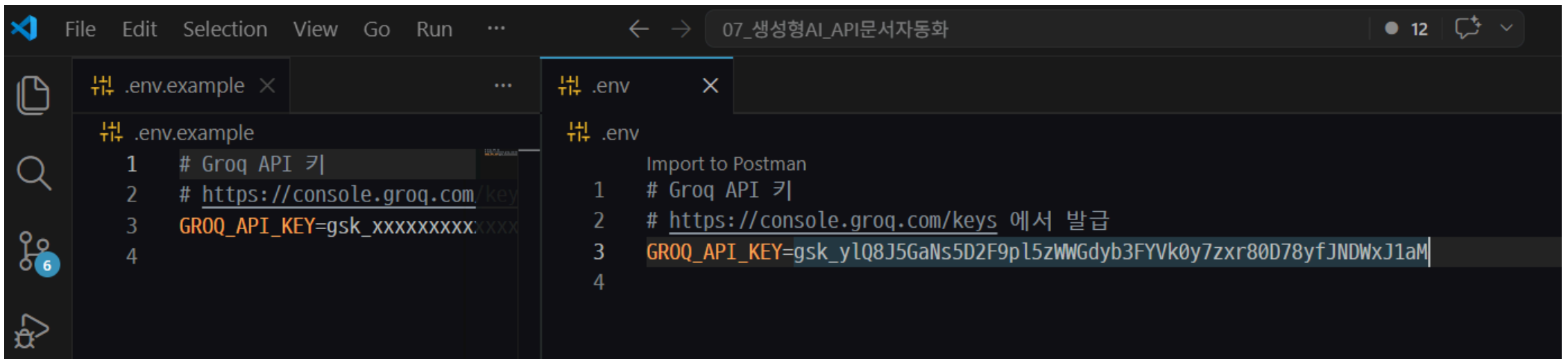
This key will not expire.

 Submit

복사



.env 파일 생성 후 붙여넣기



The screenshot shows the Visual Studio Code interface with two files open: `.env.example` and `.env`. The `.env.example` file contains the following content:

```
1 # Groq API 키
2 # https://console.groq.com/keys
3 GROQ_API_KEY=gsk_XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX
4
```

The `.env` file contains the following content:

```
Import to Postman
1 # Groq API 키
2 # https://console.groq.com/keys 에서 발급
3 GROQ_API_KEY=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
4
```

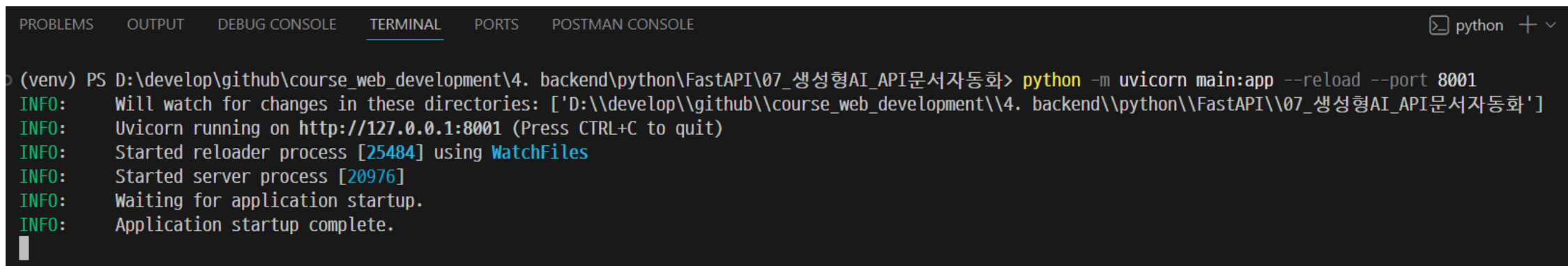
The third line of the `.env` file is highlighted, showing the copied API key.

가상환경 생성

```
# pyproject.toml과 uv.lock 기준으로 의존성 설치  
uv sync
```

서버 실행

```
python -m uvicorn main:app --reload --port 8001
```



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS POSTMAN CONSOLE python + v  
D:\develop\github\course_web_development\4. backend\python\FastAPI\07_생성형AI_API문서자동화> python -m uvicorn main:app --reload --port 8001  
INFO: Will watch for changes in these directories: ['D:\\develop\\github\\course_web_development\\4. backend\\python\\FastAPI\\07_생성형AI_API문서자동화']  
INFO: Uvicorn running on http://127.0.0.1:8001 (Press CTRL+C to quit)  
INFO: Started reloader process [25484] using WatchFiles  
INFO: Started server process [20976]  
INFO: Waiting for application startup.  
INFO: Application startup complete.
```

Swagger UI 실습 (<http://localhost:8001/docs>)

각 API에서 Try it out을 누른 뒤 아래 순서대로 입력합니다.

실습 1. 서버 상태와 OpenAPI 원본 확인

GET /health

Health 서버 상태 확인



GET /health 서버 상태 확인



Parameters

Cancel

No parameters

Execute

확인할 것

- 응답 상태 코드가 200 OK 입니다.
- 응답에 status, version, timestamp 가 들어옵니다.

Code	Details
200	Response body <pre>{ "status": "ok", "version": "1.0.0", "timestamp": "2026-07-05T12:24:49.158745" }</pre>  Download Response headers <pre>content-length: 74 content-type: application/json date: Sun, 05 Jul 2026 03:24:48 GMT server: uvicorn</pre>

실습 2. 상품 목록 조회와 Query Parameter 확인

GET /products

GET

/products 상품 목록 조회

^

등록된 상품을 카테고리 필터와 페이지네이션으로 조회합니다.

Parameters

Cancel

Name	Description
category string (string null) (query)	카테고리 필터 category
page integer (query)	페이지 번호 1 minimum: 1
size integer (query)	페이지당 항목 수 10 maximum: 100 minimum: 1

Execute

확인할 것

- 응답 상태 코드가 200 OK 입니다.
- 예제 상품인 무선 마우스 , 기계식 키보드 가 조회됩니다.

Code	Details
200	Response body <pre>[{ "id": 1, "name": "무선 마우스", "price": 35000, "stock": 50, "category": "전자제품", "created_at": "2024-01-10T00:00:00" }, { "id": 2, "name": "기계식 키보드", "price": 89000, "stock": 20, "category": "전자제품", "created_at": "2024-01-15T00:00:00" }]</pre>  Download

실습 3. 인증 실패 응답 확인

POST /products

아직 Authorize를 누르지 않은 상태에서 실행합니다.

```
{  
  "name": "USB-C 허브",  
  "price": 42000,  
  "stock": 30,  
  "category": "전자제품"  
}
```

POST

/products 상품 등록

🔒 ^

새 상품을 등록합니다. 관리자 권한이 필요합니다.

Parameters

Cancel

No parameters

Request body required

application/json ▼

Edit Value | Schema

```
{  "category": "전자제품",  "name": "무선 마우스",  "price": 35000,  "stock": 50}
```

Execute

확인할 것

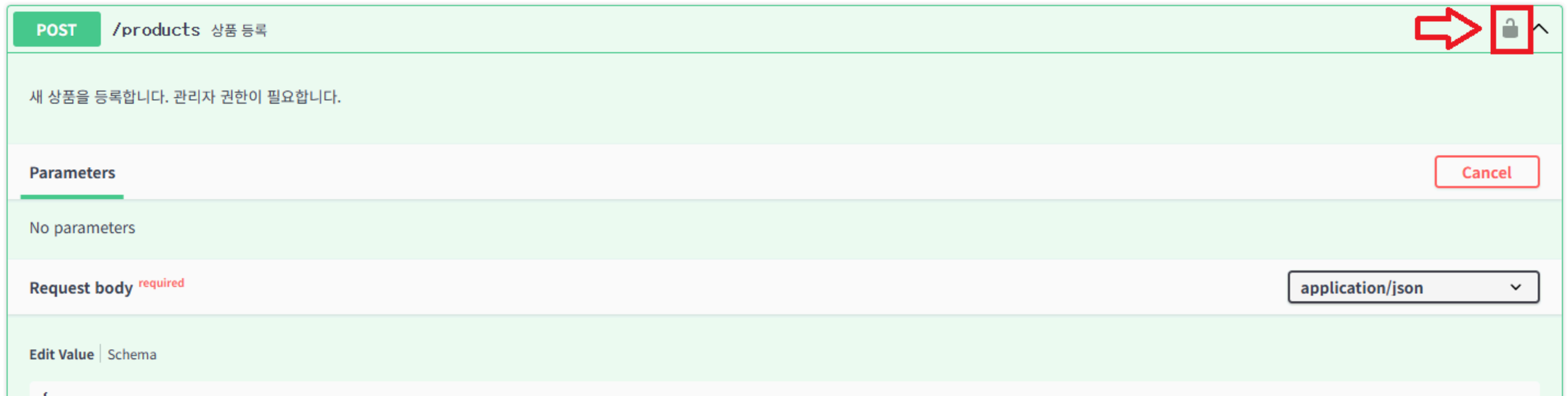
- 응답 상태 코드가 401 Unauthorized 입니다.
- 응답 본문은 {"detail": "인증이 필요합니다."} 형태입니다.

Code	Details
401	Error: Unauthorized Response body <pre>{ "detail": "인증이 필요합니다." }</pre>  Download Response headers <pre>content-length: 39 content-type: application/json date: Sun, 05 Jul 2026 03:29:41 GMT server: uvicorn</pre>

실습 4. 관리자 토큰으로 상품 등록

POST /products

1. Swagger UI 오른쪽 위 **Authorize** 버튼을 누릅니다.



POST /products 상품 등록

새 상품을 등록합니다. 관리자 권한이 필요합니다.

Parameters

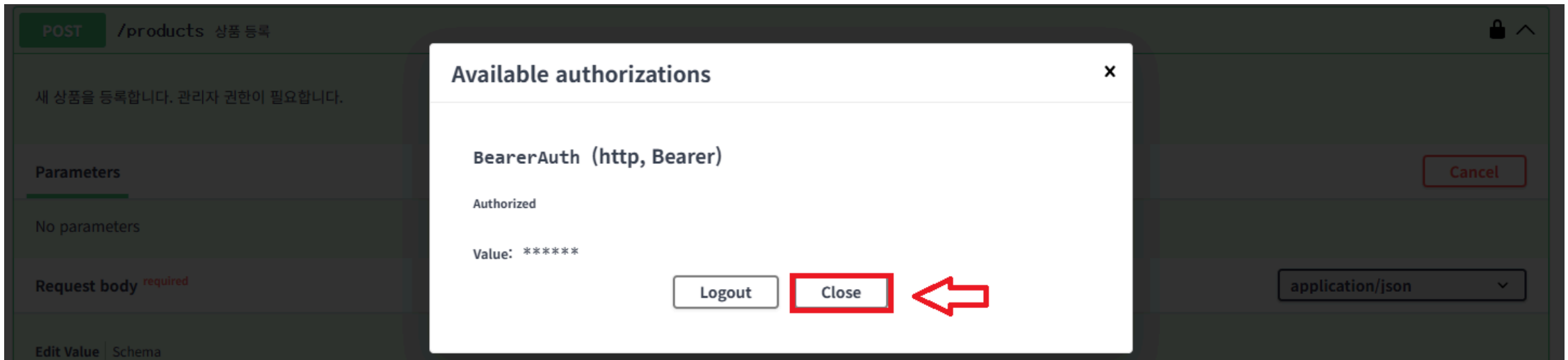
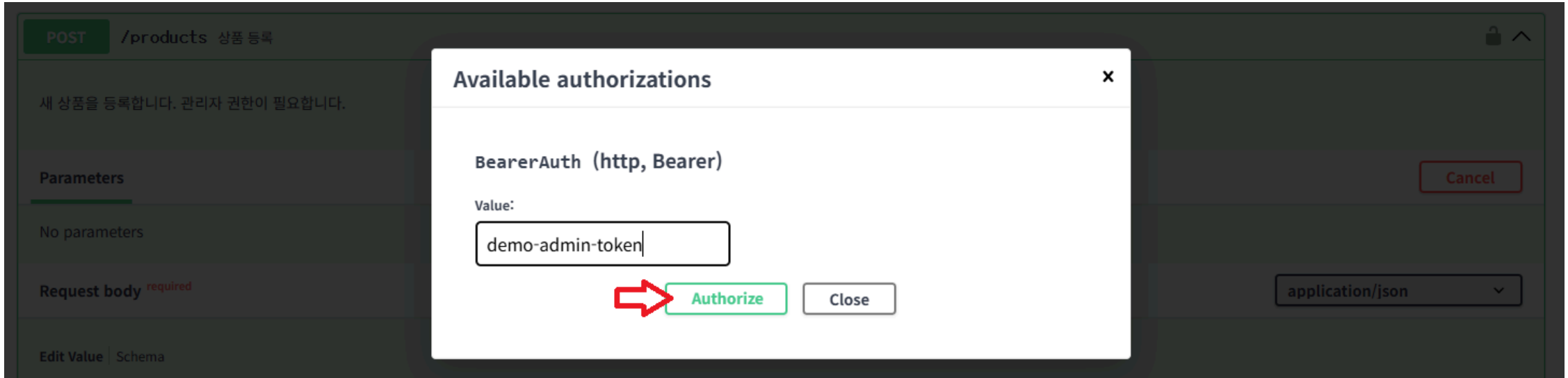
No parameters

Request body required

application/json

Edit Value | Schema

2. 값에 demo-admin-token 만 입력합니다.



3. 다시 `POST /products` 를 실행합니다.

```
{  
  "name": "USB-C 허브",  
  "price": 42000,  
  "stock": 30,  
  "category": "전자제품"  
}
```

POST

/products 상품 등록



새 상품을 등록합니다. 관리자 권한이 필요합니다.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value

Schema

```
{  "name": "USB-C 허브",  "price": 42000,  "stock": 30,  "category": "전자제품"}}
```

 Execute

확인할 것

- 응답 상태 코드가 201 Created 입니다.
- 응답에 새 상품 id 가 들어옵니다.

Code	Details
201	Response body <pre>{ "id": 3, "name": "USB-C 허브", "price": 42000, "stock": 30, "category": "전자제품", "created_at": "2026-07-05T12:35:26.343604" }</pre>  Download Response headers <pre>content-length: 123 content-type: application/json date: Sun, 05 Jul 2026 03:35:26 GMT server: unicorn</pre>

실습 5. 요청 검증 오류 확인

POST /products

```
{  
  "name": "A",  
  "price": 0,  
  "stock": -1,  
  "category": "전자제품"  
}
```

POST

/products 상품 등록

🔒 ^

새 상품을 등록합니다. 관리자 권한이 필요합니다.

Parameters

Cancel

Reset

No parameters

Request body required

application/json ▾

Edit Value | Schema

```
{  "name": "A",  "price": 0,  "stock": -1,  "category": "전자제품"}
```

Execute

확인할 것

- 응답 상태 코드가 422 Unprocessable Entity 입니다.
- name , price , stock 필드 검증 오류가 detail 목록으로 표시됩니다.

Code	Details
422	Error: Unprocessable Entity Response body <pre>{ "detail": [{ "type": "string_too_short", "loc": ["body", "name"], "msg": "String should have at least 2 characters", "input": "A", "ctx": { "min_length": 2 } }, { "type": "greater_than", "loc": [</pre>

실습 6. 상품 수정과 삭제

```
PATCH /products/{product_id}
```

`product_id` 에는 실습 4에서 생성된 상품 ID를 넣습니다.

```
{  
  "price": 39900,  
  "stock": 25  
}
```

PATCH

/products/{product_id} 상품 부분 수정



상품 정보를 부분적으로 수정합니다. 전달된 필드만 업데이트되며 관리자 권한이 필요합니다.

Parameters

Cancel

Reset

Name	Description
product_id ★ required integer (path)	1

Request body required

application/json 

Edit Value | Schema

```
{  "price": 39900,  "stock": 25}
```

Execute

확인할 것

- 응답 상태 코드가 200 OK 입니다.
- 전달한 필드만 변경됩니다.

Code	Details
200	Response body <pre>{ "id": 1, "name": "무선 마우스", "price": 39900, "stock": 25, "category": "전자제품", "created_at": "2024-01-10T00:00:00" }</pre>  Download Response headers <pre>content-length: 120 content-type: application/json date: Sun, 05 Jul 2026 03:39:37 GMT server: uvicorn</pre>

삭제도 같은 상품 ID로 확인합니다.

```
DELETE /products/{product_id}
```

DELETE /products/{product_id} 상품 삭제

🔒 ^

상품을 삭제합니다. 삭제된 상품은 복구할 수 없으며 관리자 권한이 필요합니다.

Parameters Cancel

Name	Description
product_id * required integer (path)	1

Execute

확인할 것

- 응답 상태 코드가 204 No Content 입니다.

Responses

Curl

```
curl -X 'DELETE' \  
  'http://localhost:8001/products/1' \  
  -H 'accept: */*' \  
  -H 'Authorization: Bearer demo-admin-token'
```

Request URL

```
http://localhost:8001/products/1
```

Server response

Code	Details
204	Response headers<pre>content-type: application/json date: Sun, 05 Jul 2026 03:42:01 GMT server: uvicorn</pre>

실습 7. 주문 생성

POST /orders

```
{
  "items": [
    {
      "product_id": 2,
      "quantity": 2
    }
  ]
}
```

POST

/orders 주문 생성

상품을 주문합니다. 재고 확인 후 자동으로 재고가 차감됩니다.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value

Schema

```
{  "items": [    {      "product_id": 2,      "quantity": 2    }  ]}
```

Execute

확인할 것

- 응답 상태 코드가 201 Created 입니다.
- 응답에 id, items, total_price, status, created_at 이 들어옵니다.

Code

Details

201

Response body

```
{
  "id": 1,
  "items": [
    {
      "product_id": 2,
      "name": "기계식 키보드",
      "quantity": 2,
      "unit_price": 89000
    }
  ],
  "total_price": 178000,
  "status": "pending",
  "created_at": "2026-07-05T15:10:58.539931"
}
```

 Download

실습 8. 회원 가입과 내 정보 조회

POST /users

```
{  
  "username": "hong123",  
  "email": "hong@example.com",  
  "password": "pass1234"  
}
```

POST

/users 회원 가입

^

Parameters

Cancel

Reset

No parameters

Request body required

application/json ▼

Edit Value | Schema

```
{
  "username": "hong123",
  "email": "hong@example.com",
  "password": "pass1234"
}
```

Execute

확인할 것

- 응답 상태 코드가 201 Created 입니다.
- 응답에는 password 가 포함되지 않습니다.

Code

Details

201

Response body

```
{
  "id": 1,
  "username": "hong123",
  "email": "hong@example.com",
  "created_at": "2026-07-05T15:19:42.241251"
}
```

 [Download](#)

Response headers

```
content-length: 98
content-type: application/json
date: Sun, 05 Jul 2026 06:19:41 GMT
server: uvicorn
```

이어서 관리자 토큰이 들어간 상태로 실행합니다.

GET /users/me

GET

/users/me 내 정보 조회

🔒 ^

Bearer 토큰으로 인증 후 현재 사용자 정보를 반환합니다.

Parameters

Cancel

No parameters

Execute

확인할 것

- 응답 상태 코드가 200 OK 입니다.
- 실습용 사용자 demo_user 가 반환됩니다.

Code

Details

200

Response body

```
{  
  "id": 1,  
  "username": "demo_user",  
  "email": "demo@example.com",  
  "created_at": "2026-07-05T15:20:59.366179"  
}
```

 [Download](#)

Response headers

```
content-length: 100  
content-type: application/json  
date: Sun, 05 Jul 2026 06:20:58 GMT  
server: uvicorn
```

실습 9. API 문서 생성

```
POST /ai-docs/api-docs
```

```
{  
  "server_url": "http://localhost:8001"  
}
```

POST

/ai-docs/api-docs API 문서 생성

OpenAPI 스펙을 Groq에 전달해 개발자용 마크다운 API 문서를 생성합니다.

Parameters

Cancel

No parameters

Request body required

application/json

Edit Value | Schema

```
{
  "server_url": "http://localhost:8001"
}
```

Execute

확인할 것

- `.env` 의 `GROQ_API_KEY` 가 정상이라면 응답 상태 코드가 `200 OK` 입니다.
- 응답에는 `endpoint_count` , `filename` , `path` , `content` 가 들어옵니다.

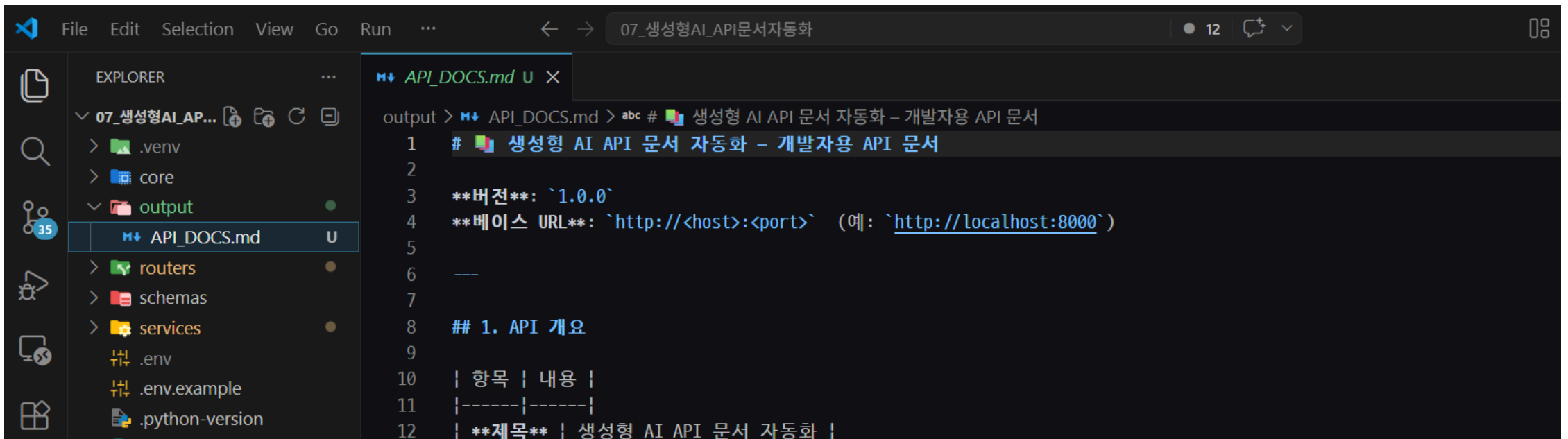
Code **Details**

200

Response body

```
{
  "endpoint_count": 15,
  "filename": "API_DOCS.md",
  "path": "output\\API_DOCS.md",
  "content": "# 🐼 생성형 AI API 문서 자동화 - 개발자용 API 문서\n\n\n**배전**: `1.0.0` \n\n**베이스 URL**: `http://<host>:<port>` (예: `http://localhost:8000`) \n\n\n# 1. API 개요\n\n| 항목 | 내용 |\n|-----|-----|\n\n**제목** | 생성형 AI API 문서 자동화 |\n\n**설명** | FastAPI 로 만든 OpenAPI 스펙을 Groq에 전달해 **API 문서**, **Postman Collection**, **README** 초안을 자동 생성하는 솔루션 서버. 쇼필름 **상품·주문·회원** 엔드포인트는 예시 API이며, `/ai-docs/*` 엔드포인트가 실제 AI 문서 생성 기능을 담당합니다. |\n\n**배전** | `1.0.0` |\n\n**베이스 URL** | `http://<host>:<port>` (예: `http://localhost:8000`) \n\n\n인증 방법 \n\n**Bearer 토큰** (*Authorization: Bearer <token>*) 을 HTTP Header에 포함해야 합니다. \n\n- 보호된 엔드포인트에만 필요합니다. \n\n- **실습용 관리자 토큰**: `demo-admin-token` (관리자 권한이 필요한 `/products` POST, PATCH, DELETE, `/users/me`, `/ai-docs/*` 등)\n\n\n**주의**: 실제 서비스에서는 안전한 JWT 를 사용하고, 토큰 검증 로직을 구현해야 합니다.\n\n\n# 3. 엔드포인트 요약\n\n| 메서드 | 경로 | 설명 |\n|-----|-----|-----|\n\n| GET | /health | 서버 상태 확인 |\n| GET | /products | 상품 목록 조회 (필터·페이지) |\n| POST | /products | **관리자** - 새 상품 등록 |\n| GET | /products/{product_id} | 단일 상품 상세 조회 |\n| PATCH | /products/{product_id} | **관리자** - 상품 부분 수정 |\n| DELETE | /products/{product_id} | **관리자** - 상품 삭제 |\n| POST | /orders | 주문 생성 (재고 차감) |\n| GET | /orders/{order_id} | 주문 상세 조회 |\n| POST | /users | 회원 가입 |\n| GET | /users/me | **인증 필요** - 현재 사용자 정보 조회 |\n| POST | /ai-docs/api-docs | OpenAPI → 마크다운 API 문서 생성 |\n| POST | /ai-docs/postman-collection | OpenAPI → Postman Collection(v2.1) 생성 |\n| POST | /ai-docs/readme | OpenAPI → GitHub README 초안 생성 |\n| POST | /ai-docs/generate-all | API 문서·Postman·README 한 번에 생성 |\n| POST | /ai-docs/changes | 변경된 엔드포인트만 문서화 |\n\n\n# 4. 엔드포인트 상세\n\n\n**표준 응답 형식**\n\n<n> { \"detail\": \"사람이 읽을 수 있는 오류 설명\" }\n\n\n**GET /health** \n\n\n**설명**: 서버가 정상 동작 중인지 확인합니다. \n\n\n**요청 파라미터**: 없음 \n\n\n**cURL 예시**\n\nbash curl -X GET http://localhost:8000/health\n\n\n\n**성공 응답 (200)**\n\n{ \"status\": \"ok\", \"version\": \"1.0.0\" }\n\n\n**응답**\n\n코드 | 설명\n|-----|-----|\n\n500 | 서버 내부 오류\n\n\n# GET /products\n\n\n**설명**: 등록된 상품을 **카테고리 필터**와 **페이지네이션**으로 조회합니다. \n\n\n**요청 파라미터**\n\n| 이름 | 위치 | 타입 | 필수 | 설명 |\n|-----|-----|-----|-----|-----|\n\n| category | query | string | null | ❌ | 카테고리 필터 |\n\n| page | query | integer | >1 | ❌ | 페이지 번호 (default: 1) |\n\n| size | query | integer | 1-100 | ❌ | 페이지당 항목 수 (default: 10) |\n\n\n**cURL 예시**\n\nbash curl -G http://localhost:8000/products \\ --data-urlencode \"category=전자제품\" \\ --data-urlencode \"page=2\" \\ --data-urlencode \"size=20\"\n\n\n\n**성공 응답 (200)** - `ProductResponse` 배열\n\n{ \"items\": [\n  {\n    \"id\": 1,\n    \"name\": \"무선 마우스\",\n    \"price\": 35000,\n    \"stock\": 50,\n    \"category\": \"전자제품\",\n    \"created_at\": \"2024-01-15T10:23:45Z\"\n  },\n  {\n    \"id\": 2,\n    \"name\": \"면 티셔츠\",\n    \"price\": 19900,\n    \"stock\": 200,\n    \"category\": \"의류\",\n    \"created_at\": \"2024-01-12T08:12:30Z\"\n  }\n],\n  \"total\": 2,\n  \"page\": 1,\n  \"size\": 20\n}\n\n\n**오류 응답**\n\n| 코드 | 설명 |\n|-----|-----|\n\n422 | Validation Error (잘못된 쿼리 파라미터) |\n\n500 | 서버 내부 오류\n\n\n\n# POST /products\n\n\n**설명**: 새 상품을 등록합니다. \n\n\n**요청 본문**\n\n{\n  \"application/json\"\n}\n\n\n**응답**\n\n{ \"application/json\" }\n\n\n\n**성공 응답 (201)** - `ProductCreate`\n\n{\n  \"id\": 3,\n  \"name\": \"무선 마우스\",\n  \"price\": 35000,\n  \"stock\": 50,\n  \"category\": \"전자제품\",\n  \"created_at\": \"2024-07-05T14:20:10Z\"\n}\n\n\n\n**오류 응답**\n\n| 코드 | 설명 |\n|-----|-----|\n\n401 | 인증 토큰 없음 또는 만료 |\n\n403 | 권한 없음 (관리자 전용) |
```

- `output/API_DOCS.md` 파일이 생성됩니다.



```
07_생성형AI_API문서자동화
12
API_DOCS.md
output > API_DOCS.md > abc # 생성형 AI API 문서 자동화 - 개발자용 API 문서
1 # 생성형 AI API 문서 자동화 - 개발자용 API 문서
2
3 **버전**: `1.0.0`
4 **베이스 URL**: `http://<host>:<port>` (예: `http://localhost:8000`)
5
6 ---
7
8 ## 1. API 개요
9
10 | 항목 | 내용 |
11 |-----|-----|
12 | **제목** | 생성형 AI API 문서 자동화 |
```

실습 10. 전체 문서 생성

```
POST /ai-docs/generate-all
```

```
{  
  "server_url": "http://localhost:8001/openapi.json"  
}
```

POST

/ai-docs/generate-all 전체 문서 생성

^

API 문서, Postman Collection, README를 한 번에 생성하고 스냅샷을 저장합니다.

Parameters

Cancel

Reset

No parameters

Request body required

application/json

Edit Value

Schema

```
{  "server_url": "http://localhost:8001/openapi.json"}

```

Execute

확인할 것

- 응답의 `files` 배열에 위 파일 경로가 들어옵니다.
- `server_url` 은 서버 기본 URL과 `/openapi.json` 전체 URL을 모두 받을 수 있습니다.

Code	Details
200	Response body <pre>{ "endpoint_count": 15, "files": [{ "filename": "API_DOCS.md", "path": "output\\API_DOCS.md", "description": "개발자용 API 문서" }, { "filename": "collection.json", "path": "output\\collection.json", "description": "Postman Import용 Collection" }, { "filename": "README.md", "path": "output\\README.md", "description": "GitHub README 초안" }], "message": "문서 생성이 완료되었습니다." }</pre>  Download

생성되는 파일:

파일	설명
output/API_DOCS.md	개발자용 API 문서
output/collection.json	Postman Import용 Collection
output/README.md	GitHub README 초안
output/_spec_snapshot.json	변경 비교용 OpenAPI 스냅샷

```
07_생성형AI_API문서자동화 12
```

EXPLORER

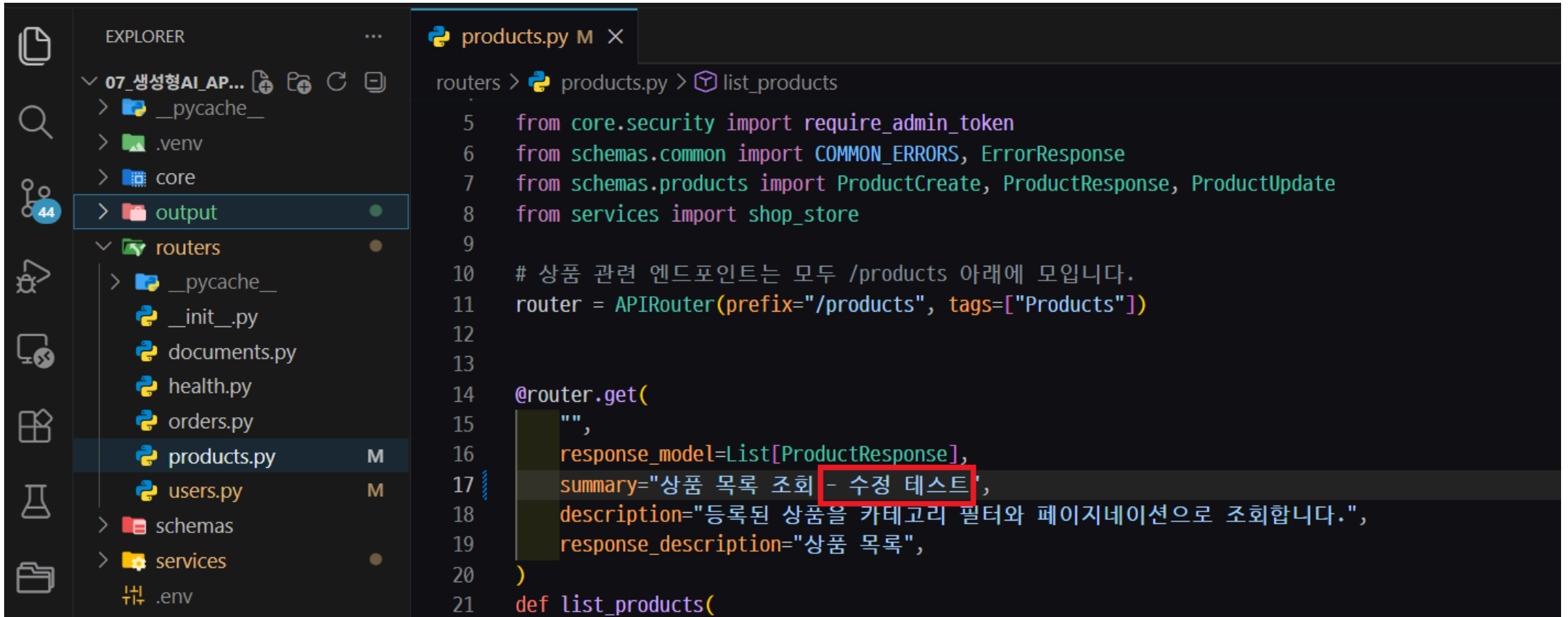
- 07_생성형AI_AP...
 - .venv
 - core
 - output
 - _spec_snapshot.json**
 - API_DOCS.md
 - collection.json
 - README.md
 - routers
 - schemas
 - services
 - .env
 - .env.example
 - .python-version

output > _spec_snapshot.json > ...

```
1  {
2    "openapi": "3.1.0",
3    "info": {
4      "title": "생성형 AI API 문서 자동화",
5      "summary": "FastAPI OpenAPI 스펙을 Groq로 문서화하는 실습 API",
6      "description": "\n## 개요\n\n이 API는 FastAPI가 만든 OpenAPI 스펙을 생성형 AI에 전달해\nAPI 문서, Postman Collection, README",
7      "version": "1.0.0"
8    },
9    "paths": {
10     "/health": {
11       "get": {
12         "tags": [
13           "Health"
14         ],
15         "summary": "서버 상태 확인",
```

실습 11. 변경 문서 생성

1. `routes/products.py` 의 `summary` 나 `description` 문구를 잠깐 수정합니다.



```
EXPLORER
07_생성형AI_AP...
  > __pycache__
  > .venv
  > core
  > output
  > routes
    > __pycache__
    > __init__.py
    > documents.py
    > health.py
    > orders.py
    > products.py M
    > users.py M
  > schemas
  > services
  > .env

products.py M X
routes > products.py > list_products
5 from core.security import require_admin_token
6 from schemas.common import COMMON_ERRORS, ErrorResponse
7 from schemas.products import ProductCreate, ProductResponse, ProductUpdate
8 from services import shop_store
9
10 # 상품 관련 엔드포인트는 모두 /products 아래에 모입니다.
11 router = APIRouter(prefix="/products", tags=["Products"])
12
13
14 @router.get(
15     "",
16     response_model=List[ProductResponse],
17     summary="상품 목록 조회 - 수정 테스트",
18     description="등록된 상품을 카테고리 필터와 페이지네이션으로 조회합니다.",
19     response_description="상품 목록",
20 )
21 def list_products(
```

2. 서버가 reload된 뒤 `POST /ai-docs/changes` 를 다시 실행합니다.

`POST /ai-docs/changes`

스냅샷 없이 먼저 실행해 봅니다.

```
{  
  "server_url": "http://localhost:8001"  
}
```

POST

/ai-docs/changes 변경된 엔드포인트 문서화

^

이전 OpenAPI 스냅샷과 현재 스펙을 비교해 변경된 엔드포인트만 문서화합니다.

Parameters

Cancel

No parameters

Request body required

application/json

▼

Edit Value | Schema

```
{  "server_url": "http://localhost:8001"}
```

//

Execute

3. 변경이 있으면 output/API_CHANGES.md 가 생성됩니다.

Code

Details

200

Response body

```
{
  "endpoint_count": 15,
  "filename": "API_CHANGES.md",
  "path": "output\\API_CHANGES.md",
  "content": "# API 변경사항 문서 (Markdown)\n\n> **변경** `vX.Y.Z` (변경 적용일: 2026-07-05) \n\n> **대상** `GET /products` (상품 목록 조회)\n\n---\n\n# 변경 요약\n\n구분 | 건수\n-----|-----\n| \n| **추가** (New) | 0\n| **수정** (Modified) | 1\n| **삭제** (Deleted) | 0\n| **전체** | 1\n\n> **핵심** `GET /products` 엔드포인트가 **수정**되었습니다. \n\n> 기존 `summary` 가 `상품 목록 조회` → `상품 목록 조회 - 수정 테스트` 로 변경 \n\n> `description` 에 `등록된 상품을 카테고리 필터와 페이지네이션으로 조회합니다.` 라는 상세 설명이 추가되었습니다.\n\n> - 파라미터 정의는 그대로 유지되지만, `category` 파라미터에 `null` 허용이 명시되었습니다. \n\n---\n\n# 각 변경 엔드포인트 상세\n\nHTTP Method | Path | 구분 | 주요 변경 내용\n-----|-----|-----|-----\n\nGET **`GET /products`** \n\n**수정** \n\n- `summary` 와 `description` 업데이트 \n\n- `category` 파라미터에 `null` 허용 (`anyOf: string | null`) \n\n- 응답 스키마는 기존 `ProductResponse` 배열 유지 (변경 없음)\n\n2.1 `GET /products` (수정)\n\n항목 | 내용\n-----|-----\n\n| **태그** | `Products`\n\n| **Operation ID** | `list_products_get`\n\n| **요약** | `상품 목록 조회 - 수정 테스트`\n\n| **설명** | `등록된 상품을 카테고리 필터와 페이지네이션으로 조회합니다.`\n\n| **쿼리 파라미터** | `category` (string | null, optional) - 카테고리 필터 \n\n- `page` (integer, ≥1, default = 1) - 페이지 번호 \n\n- `size` (integer, 1-100, default = 10) - 페이지당 항목 수\n\n| **응답** | `200 OK` - `application/json` \n\n→ `ProductResponse[]`\n\n<br> `422 Unprocessable Entity` - `HTTPValidationError`\n\n---\n\n# `curl` 예시 및 응답 예시\n\n> **※ 아래 예시는 **수정된** 엔드포인트에 대한 요청/응답을 보여줍니다.**\n\n3.1 요청 예시 (curl)\n\n`bash\ncurl -X GET \"https://api.example.com/products?category=electronics&page=2&size=5\" -H \"Accept: application/json\"\n\n> -H `Accept: application/json`\n\n> `category=electronics` : `electronics` 카테고리만 조회 \n\n> `page=2` : 두 번째 페이지 \n\n> `size=5` : 페이지당 5개 항목 반환\n\n> **Tip** - `category` 파라미터에 값을 주지 않으면 모든 카테고리가 포함됩니다. `null` 로 명시하고 싶을 경우 `category=` 혹은 `category=null` 로 전달하면 됩니다.\n\n3.2 성공 응답 예시 (`200 OK`)\n\n`json`\n\n{\n  \"id\": 101,\n  \"name\": \"Wireless Bluetooth Headphones\",\n  \"category\": \"electronics\",\n  \"price\": 79.99,\n  \"currency\": \"USD\",\n  \"stock\": 34,\n  \"description\": \"Noise-cancelling over-ear headphones with 20 h battery life.\",\n  \"created_at\": \"2024-11-12T08:45:30Z\",\n  \"updated_at\": \"2026-06-20T14:12:07Z\"\n},\n{\n  \"id\": 102,\n  \"name\": \"Smart LED Light Bulb\",\n  \"category\": \"electronics\",\n  \"price\": 19.5,\n  \"currency\": \"USD\",\n  \"stock\": 120,\n  \"description\": \"Wi-Fi enabled bulb, supports color temperature adjustment.\",\n  \"created_at\": \"2025-02-03T11:20:10Z\",\n  \"updated_at\": \"2026-05-15T09:33:44Z\"\n},\n{\n  \"id\": 103,\n  \"name\": \"Portable Power Bank 2000mAh\",\n  \"category\": \"electronics\",\n  \"price\": 45.0,\n  \"currency\": \"USD\",\n  \"stock\": 58,\n  \"description\": \"Fast-charge power bank with dual USB-C output.\",\n  \"created_at\": \"2025-09-27T16:02:55Z\",\n  \"updated_at\": \"2026-04-01T07:18:22Z\"\n},\n{\n  \"id\": 104,\n  \"name\": \"4K Ultra HD Action Camera\",\n  \"category\": \"electronics\",\n  \"price\": 129.99,\n  \"currency\": \"USD\",\n  \"stock\": 22,\n  \"description\": \"Water-proof action cam with 60 fps 4K recording.\",\n  \"created_at\": \"2026-01-14T13:44:08Z\",\n  \"updated_at\": \"2026-06-30T12:00:00Z\"\n},\n{\n  \"id\": 105,\n  \"name\": \"Bluetooth Speaker Mini\",\n  \"category\": \"electronics\",\n  \"price\": 34.75,\n  \"currency\": \"USD\",\n  \"stock\": 77,\n  \"description\": \"Compact speaker with 10 h playtime and built-in mic.\",\n  \"created_at\": \"2025-07-19T09:10:41Z\",\n  \"updated_at\": \"2026-06-10T18:22:13Z\"\n}\n}\n\n> **오류 응답 예시** (`422 Unprocessable Entity`)\n\n`json`\n\n{\n  \"detail\": [\n    {\n      \"loc\": [\"query\", \"size\"],\n      \"msg\": \"ensure this value is less than or equal to 100\", \n      \"type\": \"value_error.number.not_le\", \n      \"ctx\": { \"limit_value\": 100 }\n    }\n  ]\n}\n\n> **설명** - `size` 파라미터가 허용 범위를 초과했습니다.\n\n---\n\n# 참고: `ProductResponse` 스키마 (요약)\n\n필드 | 타입 | 설명\n-----|-----|-----\n`id` | integer | 상품 고유 식별자\n`name` | string | 상품명
```

Download

오류 상황 읽기

상태 코드	상황	확인할 것
400	주문 재고 부족 또는 변경 비교용 스냅샷 없음	재고 수량 또는 <code>POST /ai-docs/generate-all</code> 실행 여부 확인
401	Bearer 토큰 누락	Swagger Authorize에 <code>demo-admin-token</code> 입력
403	Bearer 토큰 값 오류	<code>Bearer</code> 접두어 없이 토큰 문자열만 입력

상태 코드	상황	확인할 것
404	존재하지 않는 상품/주문 ID	경로 파라미터 ID 확인
409	상품명 또는 이메일 중복	다른 <code>name</code> 또는 <code>email</code> 사용
422	요청 본문/파라미터 검증 실패	<code>Field</code> 의 <code>min_length</code> , <code>gt</code> , <code>ge</code> 조건 확인
500	서버 내부 오류	Swagger의 공통 오류 응답 문서 확인
502	OpenAPI 스펙 수집 또는 Groq 호출 실패	서버 주소, <code>.env</code> , Groq 사용량 제한 확인